Contents

LAB 1. Normalization 2
1.1 Implementation of Standard scaler (normalize data with standard scaler) 2
1.2 Implementation of min-max scaler. 2
LAB 2 : Data Cleaning 3
2.1 : Sample code to do Data Cleaning 3
2.2 : Sample code to do Data Cleaning 4
LAB 3 : Data Transformation (DOB to AGE): 6
LAB 4 : Association Rule Mining (ARM) 7
4.1 Association-rule-mining-via-apriori-algorithm 7
4.2 Sample code to do FP-Growth in Python 10
LAB 5 : Classification (Using Naive Bayes Classifier) 11
Lab 6: Clustering 12
6.1 : K-Means Clustering 12
6.2 Mini-Batch K-means Clustering 14
6.3 Hierarchical Clustering (Agglomerative Clustering) 16
6.4 Clustering Iris data using KMedoids. 17

LAB 1. Normalization

1.1 Implementation of Standard scaler (normalize data with standard scaler)

Definition:
Standard Scaler is a normalization technique that transforms data so each feature has a mean of 0 and a standard deviation of 1.It removes the effect of different scales between variables, making them comparable.This process is also called Z-score normalization and is widely used in machine learning.

Formula:

$Z = (x-\mu) / \sigma$, where x = data value, μ = column mean, σ = column standard deviation.

Code:

```
import numpy as np
import pandas as pd
class StandardNorm:
  def scale(self,d):
    for i in d.columns:
      mean=d[i].mean()
      sd=d[i].std()
      d[i]=(d[i]-mean)/sd
    return d
data=pd.DataFrame([[45000,42],[32000,26],[58000,48],[37000,32]],columns=['Salary','Age'])
print("Original Data")
print(data)
s=StandardNorm()
df=s.scale(data)
print("\nScaled Data")
print(df)
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilastandard_scalar.py
Original Data
   Salary  Age
0   45000   42
1   32000   26
2   58000   48
3   37000   32

Scaled Data
     Salary       Age
0  0.176318  0.506803
1 -0.969750 -1.114967
2  1.322386  1.114967
3 -0.528954 -0.506803
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab>
```

1.2 Implementation of min-max scaler.

Definition : Min-Max Scaler is a normalization technique that rescales features into a fixed range, usually [0, 1].This method is useful when all features should have the same scale for fair comparison.

Formula: $x' = (x-x_{min}) / (x_{max}-x_{min})$

where x = original value, xmin = minimum of column, xmax = maximum of column.

Code:

```
import pandas as pd
class MinMaxNorm:
  def scale(self,d):
    for i in d.columns:
      min=d[i].min()
      max=d[i].max()
      d[i]=(d[i]-min)/(max-min)
    return d
data=pd.DataFrame([[45000,42],[32000,26],[58000,48],[37000,32]],columns=['Salary','Age'])
print("Original Data")
print(data)
s=MinMaxNorm()
print("\nScaled Data")
df=s.scale(data)
print(df)
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilaminmax_scalar.py
Original Data
   Salary  Age
0   45000   42
1   32000   26
2   58000   48
3   37000   32

Scaled Data
     Salary       Age
0  0.500000  0.727273
1  0.000000  0.000000
2  1.000000  1.000000
3  0.192308  0.272727
```

LAB 2 : Data Cleaning

2.1 : Sample code to do Data Cleaning

Theory: Data Cleaning is the process of detecting and correcting inaccurate, incomplete, or irrelevant data to improve dataset quality. It includes handling missing values, duplicates, outliers, and inconsistent formatting. Clean data ensures reliable results in data analysis and machine learning models.

Common Steps in Data Cleaning

1. **Load data** → Read CSV/Excel/DB files.
2. **Check dataset info** → data types, missing values, shape.
3. **Handle missing values** → drop or fill with mean/median/mode.
4. **Remove duplicates** → drop repeated rows.
5. **Fix inconsistent data** → formatting, capitalization, typos.
6. **Handle outliers** → remove or transform extreme values.

Code:

```
import pandas as pd
import numpy as np
data = pd.read_csv('employees.csv')
print("Original Data")
print(data[0:25])
# Removing missing values, duplicate rows & column Boonus %
data=data.dropna(axis=0)
data.drop_duplicates(keep='first',inplace=True)
del data['Bonus %']

# Correcting Inconsitencies among values

data['Team']=data['Team'].str.replace('Fin','Finance')

data['Team']=data['Team'].str.replace('Mkt','Marketing')

data['Team']=data['Team'].str.replace('Financeance','Finance')

print("Cleaned Data")

print(data[0:25])

data.to_csv('employees_cleaned.csv', index=False)

print("Successfully Cleaned...")
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopiladatacleaning.py
Original Data
   First Name  Gender    Salary  Bonus %                  Team
0     Douglas    Male   97308.0    6.945             Marketing
1     Douglas    Male   97308.0    6.945             Marketing
2     Douglas    Male   97310.0    6.945             Marketing
3      Thomas    Male   61933.0    4.170                   NaN
4       Maria  Female  130590.0   11.858               Finance
5       Jerry    Male  138705.0    9.340                   Fin
6       Larry    Male       NaN    1.389       Client Services
7      Dennis    Male  115163.0   10.125                 Legal
8        Ruby  Female   65476.0   10.012               Product
9         NaN  Female   45906.0   11.598                   Fin
18    Lillian  Female   59414.0    1.256               Product
19     Jeremy    Male   90370.0    7.369       Human Resources
20      Shawn    Male  111737.0    6.414               Product
21      Diana  Female  132940.0   19.082       Client Services
22      Donna  Female   81014.0    1.894               Product
23       Lois     NaN   64714.0    4.934                 Legal
24    Matthew    Male  100612.0   13.645                   Mkt
Cleaned Data
   First Name  Gender    Salary                  Team
0     Douglas    Male   97308.0             Marketing
2     Douglas    Male   97310.0             Marketing
4       Maria  Female  130590.0               Finance
5       Jerry    Male  138705.0               Finance
7      Dennis    Male  115163.0                 Legal
8        Ruby  Female   65476.0               Product
10     Angela  Female   95570.0           Engineering
12    Frances  Female  139852.0  Business Development
14      Julie  Female  102508.0                 Legal
15    Brandon    Male  112807.0       Human Resources
16       Gary    Male  109831.0                 Sales
18    Lillian  Female   59414.0               Product
19     Jeremy    Male   90370.0       Human Resources
20      Shawn    Male  111737.0               Product
21      Diana  Female  132940.0       Client Services
22      Donna  Female   81014.0               Product
24    Matthew    Male  100612.0             Marketing
27       John    Male   97950.0       Client Services
29      Craig    Male   37598.0             Marketing
31      Terry    Male  124008.0       Client Services
32   Benjamin    Male   79529.0                 Legal
33  Christina  Female  118780.0           Engineering
36       Jean  Female  119082.0  Business Development
37      Jerry    Male   95734.0       Client Services
38    Theresa  Female   85182.0                 Sales
Successfully Cleaned...
```

2.2 : Sample code to do Data Cleaning

Code:

```
import pandas as pd
import numpy as np
data = pd.read_csv('employees.csv')
print("Original Data")
print(data[0:25])
# Filling missing values with mean
data['Salary']=data['Salary'].fillna(data['Salary'].mean())
print("Cleaned Data")
print(data[0:25])
data = pd.read_csv('employees.csv')
print("Original Data")
print(data[0:25])
# Filling missing values with i
data['Salary']=data['Salary'].interpolate(method="linear")
print("Cleaned Data")
print(data[0:25])
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopiladataclean2.py
Original Data
   First Name  Gender    Salary  Bonus %                  Team
0      Douglas    Male   97308.0    6.945             Marketing
1      Douglas    Male   97308.0    6.945             Marketing
2      Douglas    Male   97310.0    6.945             Marketing
3       Thomas    Male   61933.0    4.170                   NaN
4        Maria  Female  130590.0   11.858               Finance
5        Jerry    Male  138705.0    9.340                   Fin
6        Larry    Male       NaN    1.389       Client Services
7       Dennis    Male  115163.0   10.125                 Legal
8         Ruby  Female   65476.0   10.012               Product
9          NaN  Female   45906.0   11.598                   Fin
10      Angela  Female   95570.0   18.523           Engineering
11       Maria  Female  130590.0   11.858               Finance
12     Frances  Female  139852.0    7.524  Business Development
13      Louise  Female   63241.0   15.132                   NaN
14       Julie  Female  102508.0   12.637                 Legal
15     Brandon    Male  112807.0   17.492       Human Resources
16        Gary    Male  109831.0    5.831                 Sales
17    Kimberly  Female       NaN   14.543               Finance
18     Lillian  Female   59414.0    1.256               Product
19      Jeremy    Male   90370.0    7.369       Human Resources
20       Shawn    Male  111737.0    6.414               Product
21       Diana  Female  132940.0   19.082       Client Services
22       Donna  Female   81014.0    1.894               Product
23         Lois     NaN   64714.0    4.934                 Legal
24     Matthew    Male  100612.0   13.645                   Mkt
Cleaned Data
   First Name  Gender         Salary  Bonus %                  Team
0      Douglas    Male   97308.000000    6.945             Marketing
1      Douglas    Male   97308.000000    6.945             Marketing
2      Douglas    Male   97310.000000    6.945             Marketing
3       Thomas    Male   61933.000000    4.170                   NaN
4        Maria  Female  130590.000000   11.858               Finance
5        Jerry    Male  138705.000000    9.340                   Fin
6        Larry    Male   90754.204795    1.389       Client Services
7       Dennis    Male  115163.000000   10.125                 Legal
8         Ruby  Female   65476.000000   10.012               Product
9          NaN  Female   45906.000000   11.598                   Fin
10      Angela  Female   95570.000000   18.523           Engineering
11       Maria  Female  130590.000000   11.858               Finance
12     Frances  Female  139852.000000    7.524  Business Development
13      Louise  Female   63241.000000   15.132                   NaN
14       Julie  Female  102508.000000   12.637                 Legal
15     Brandon    Male  112807.000000   17.492       Human Resources
16        Gary    Male  109831.000000    5.831                 Sales
17    Kimberly  Female   90754.204795   14.543               Finance
18     Lillian  Female   59414.000000    1.256               Product
19      Jeremy    Male   90370.000000    7.369       Human Resources
20       Shawn    Male  111737.000000    6.414               Product
21       Diana  Female  132940.000000   19.082       Client Services
22       Donna  Female   81014.000000    1.894               Product
23         Lois     NaN   64714.000000    4.934                 Legal
24     Matthew    Male  100612.000000   13.645                   Mkt
```

```
Original Data
   First Name  Gender     Salary  Bonus %                  Team
0      Douglas    Male    97308.0    6.945             Marketing
1      Douglas    Male    97308.0    6.945             Marketing
2      Douglas    Male    97310.0    6.945             Marketing
3       Thomas    Male    61933.0    4.170                   NaN
4        Maria  Female   130590.0   11.858               Finance
5        Jerry    Male   138705.0    9.340                   Fin
6        Larry    Male       NaN    1.389        Client Services
7       Dennis    Male   115163.0   10.125                 Legal
8         Ruby  Female    65476.0   10.012               Product
9          NaN  Female    45906.0   11.598                   Fin
10      Angela  Female    95570.0   18.523           Engineering
11       Maria  Female   130590.0   11.858               Finance
12     Frances  Female   139852.0    7.524  Business Development
13      Louise  Female    63241.0   15.132                   NaN
14       Julie  Female   102508.0   12.637                 Legal
15     Brandon    Male   112807.0   17.492        Human Resources
16        Gary    Male   109831.0    5.831                 Sales
17    Kimberly  Female       NaN   14.543               Finance
18     Lillian  Female    59414.0    1.256               Product
19      Jeremy    Male    90370.0    7.369        Human Resources
20       Shawn    Male   111737.0    6.414               Product
21       Diana  Female   132940.0   19.082        Client Services
22       Donna  Female    81014.0    1.894               Product
23        Lois     NaN    64714.0    4.934                 Legal
24     Matthew    Male   100612.0   13.645                   Mkt
Cleaned Data
   First Name  Gender     Salary  Bonus %                  Team
0      Douglas    Male    97308.0    6.945             Marketing
1      Douglas    Male    97308.0    6.945             Marketing
2      Douglas    Male    97310.0    6.945             Marketing
3       Thomas    Male    61933.0    4.170                   NaN
4        Maria  Female   130590.0   11.858               Finance
5        Jerry    Male   138705.0    9.340                   Fin
6        Larry    Male   126934.0    1.389        Client Services
7       Dennis    Male   115163.0   10.125                 Legal
8         Ruby  Female    65476.0   10.012               Product
9          NaN  Female    45906.0   11.598                   Fin
10      Angela  Female    95570.0   18.523           Engineering
11       Maria  Female   130590.0   11.858               Finance
12     Frances  Female   139852.0    7.524  Business Development
13      Louise  Female    63241.0   15.132                   NaN
14       Julie  Female   102508.0   12.637                 Legal
15     Brandon    Male   112807.0   17.492        Human Resources
16        Gary    Male   109831.0    5.831                 Sales
17    Kimberly  Female    84622.5   14.543               Finance
18     Lillian  Female    59414.0    1.256               Product
19      Jeremy    Male    90370.0    7.369        Human Resources
20       Shawn    Male   111737.0    6.414               Product
21       Diana  Female   132940.0   19.082        Client Services
22       Donna  Female    81014.0    1.894               Product
23        Lois     NaN    64714.0    4.934                 Legal
24     Matthew    Male   100612.0   13.645                   Mkt
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab>
```

LAB 3 : Data Transformation (DOB to AGE):

Theory : Data Transformation means converting raw data into a more useful format for analysis. A common transformation is converting Date of Birth (DOB) into Age, which is easier to analyze in models.This uses today's date and subtracts the birth year (considering months/days for accuracy).

Code:

```
import pandas as pd
# Read data
data = pd.read_csv('data_with_age.csv')
print("Original Data")
print(data.head())
# Convert DOB column to datetime
data["DOB"] = pd.to_datetime(data["DOB"])
# Get today's date
today = pd.to_datetime("today")
# Calculate AGE in years
data["AGE"] = ((today - data["DOB"]).dt.days / 365.25).astype(int)
# Save if needed
# data.to_csv("F:/DWDM/lab/python/data_with_age.csv", index=False)
print("Transformed Data")
print(data.to_string(index=False))
```

OR NEXT METHOD:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# New Variable
from dateutil.relativedelta import *
from datetime import *
df = pd.read_csv("data_with_age.csv")
def get_age(dob):
    now = datetime.now()
    age = relativedelta(now, dob).years
    return age
df['age'] = pd.to_datetime(df['dob']).apply(get_age)
data = df.filter(['name', 'dob', 'age', 'is_student', 'target'])
print(data)
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopiladatatransformation.py
Original Data
   ID  NAME         DOB  AGE        age
0   1   Ram  2015-02-10    8   8.000000
1   2  hari  1998-01-05   20  25.097878
Transformed Data
 ID NAME        DOB  AGE       age
  1  Ram 2015-02-10   10  8.000000
  2 hari 1998-01-05   27 25.097878
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab>
```

LAB 4 : Association Rule Mining (ARM)

Association rule mining is a technique used to **find hidden relationships or patterns** among items in a dataset.It is mostly applied in **market basket analysis** (e.g., "If a customer buys bread, they are likely to buy butter").

Key Measures:

1. **Support (X → Y)**

   Support(X→Y) = Transactions containing (X∪Y) / Total Transactions

   → How frequently items appear together.

2. **Confidence (X → Y)**

   Confidence(X→Y)=Transactions containing (X∪Y) / Transactions containing X

   → Likelihood of buying Y when X is bought.

3. **Lift (X → Y)**

   Lift(X→Y)=Confidence(X→Y)/Support(Y)

   → Strength of rule (> 1 means strong association).

4.1 Association-rule-mining-via-apriori-algorithm

Q.Let's suppose that we want rules for only those items that are purchased at least 5 times a day, or 7 x 5 = 35 times in one week, since our dataset is for a one-week time period. The support for those items can be calculated as 35/7500 = 0.0045. The minimum confidence for the rules is 20% or 0.2. Similarly, we specify the value for lift as 3 and finally min_length is 2 since we want at least two

products in our rules. These values are mostly just arbitrarily chosen, so you can play with these values and see what difference it makes in the rules you get back out.

Theory: Apriori is a classic algorithm used in association rule mining to find frequent itemsets in a transactional dataset . It works by iteratively generating candidate itemsets and pruning those that do not meet the minimum support. Once frequent itemsets are found, it derives strong association rules based on confidence and lift.

Algorithm:

1. Scan the dataset to find frequent 1-itemsets with minimum support.
2. Generate candidate k-itemsets from frequent (k-1)-itemsets.
3. Prune candidates that do not meet minimum support.
4. Repeat until no more frequent itemsets are found.
5. Generate association rules from frequent itemsets using minimum confidence.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from apyori import apriori
store_data = pd.read_csv('store_data.csv',header=None)
store_data.head()
records = []
for i in range(0, 7501):
    records.append([str(store_data.values[i,j]) for j in range(0, 20)])
association_rules = apriori(records, min_support=0.0045, min_confidence=0.2, min_lift=3,
min_length=2)
association_results = list(association_rules)
for item in association_results:
    # first index of the inner list
    # Contains base item and add item
    pair = item[0]
    items = [x for x in pair]
    print("Rule: " + items[0] + " -> " + items[1])
    #second index of the inner list
    print("Support: " + str(item[1]))
    #third index of the list located at 0th
    #of the third index of the inner list
    print("Confidence: " + str(item[2][0][2]))
    print("Lift: " + str(item[2][0][3]))
    print("=====================================")
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilaapriori.py
Rule: chicken -> light cream
Support: 0.004532728969470737
Confidence: 0.29059829059829057
Lift: 4.84395061728395
====================================
Rule: escalope -> mushroom cream sauce
Support: 0.005732568990801226
Confidence: 0.3006993006993007
Lift: 3.790832696715049
====================================
Rule: pasta -> escalope
Support: 0.005865884548726837
Confidence: 0.3728813559322034
Lift: 4.700811850163794
====================================
Rule: herb & pepper -> ground beef
Support: 0.015997866951073192
Confidence: 0.3234501347708895
Lift: 3.2919938411349285
====================================
Rule: tomato sauce -> ground beef
Support: 0.005332622317024397
Confidence: 0.3773584905660377
Lift: 3.840659481324083
====================================
Rule: olive oil -> whole wheat pasta
Support: 0.007998933475536596
Confidence: 0.2714932126696833
Lift: 4.122410097642296
====================================
Rule: pasta -> shrimp
Support: 0.005065991201173177
Confidence: 0.3220338983050847
Lift: 4.506672147735896
====================================
Rule: nan -> chicken
Support: 0.004532728969470737
Confidence: 0.29059829059829057
Lift: 4.84395061728395
====================================
Rule: chocolate -> frozen vegetables
Support: 0.005332622317024397
Confidence: 0.23255813953488375
Lift: 3.2545123221103784
====================================
Rule: ground beef -> cooking oil
Support: 0.004799360085321957
Confidence: 0.5714285714285714
Lift: 3.2819951870487856
====================================
Rule: nan -> escalope
Support: 0.005732568990801226
Confidence: 0.3006993006993007
Lift: 3.790832696715049
====================================
Rule: nan -> escalope
Support: 0.005865884548726837
Confidence: 0.3728813559322034
Lift: 4.700811850163794
====================================
Rule: frozen vegetables -> ground beef
Support: 0.008665511265164644
Confidence: 0.31100478468899523
Lift: 3.165328208890303
```

```
Rule: olive oil -> frozen vegetables
Support: 0.004799360085321957
Confidence: 0.20338983050847456
Lift: 3.088314005352364
====================================
Rule: mineral water -> frozen vegetables
Support: 0.007199040127982935
Confidence: 0.30508474576271183
Lift: 3.200616332819722
====================================
Rule: olive oil -> frozen vegetables
Support: 0.005732568990801226
Confidence: 0.20574162679425836
Lift: 3.1240241752707125
====================================
Rule: frozen vegetables -> spaghetti
Support: 0.005999200106652446
Confidence: 0.21531100478468898
Lift: 3.0131489680782684
====================================
Rule: frozen vegetables -> spaghetti
Support: 0.006665777896280496
Confidence: 0.23923444976076558
Lift: 3.4980460188216425
====================================
Rule: grated cheese -> ground beef
Support: 0.005332622317024397
Confidence: 0.3225806451612903
Lift: 3.283144395325426
====================================
Rule: herb & pepper -> mineral water
Support: 0.006665777896280496
Confidence: 0.39062500000000006
Lift: 3.975682666214383
====================================
Rule: nan -> herb & pepper
Support: 0.015997866951073192
Confidence: 0.3234501347708895
Lift: 3.2919938411349285
====================================
Rule: herb & pepper -> ground beef
Support: 0.006399146780429276
Confidence: 0.3934426229508197
Lift: 4.004359721511667
====================================
Rule: olive oil -> ground beef
Support: 0.004932675643247567
Confidence: 0.22424242424242427
Lift: 3.40494417862839
====================================
Rule: nan -> tomato sauce
Support: 0.005332622317024397
Confidence: 0.3773584905660377
Lift: 3.840659481324083
====================================
Rule: ground beef -> spaghetti
Support: 0.005999200106652446
Confidence: 0.5232558139534884
Lift: 3.005315360233627
====================================
Rule: olive oil -> spaghetti
Support: 0.007199040127982935
Confidence: 0.20300751879699247
Lift: 3.0825089038385434
```

```
Rule: olive oil -> mineral water
Support: 0.005199306759098787
Confidence: 0.22543352601156072
Lift: 3.4230301186492245
=====================================
Rule: nan -> whole wheat pasta
Support: 0.007998933475536596
Confidence: 0.2714932126696833
Lift: 4.13077198425009
=====================================
Rule: nan -> pasta
Support: 0.005065991201173177
Confidence: 0.3220338983050847
Lift: 4.515095833993347
=====================================
Rule: olive oil -> spaghetti
Support: 0.005065991201173177
Confidence: 0.20105820105820105
Lift: 3.0529100529100526
=====================================
Rule: nan -> chocolate
Support: 0.005332622317024397
Confidence: 0.23255813953488375
Lift: 3.260595522712454
=====================================
Rule: nan -> ground beef
Support: 0.004799360085321957
Confidence: 0.5714285714285714
Lift: 3.2819951870487856
=====================================
Rule: nan -> frozen vegetables
Support: 0.008665511265164644
Confidence: 0.31100478468899523
Lift: 3.165328208890303
=====================================
Rule: mineral water -> frozen vegetables
Support: 0.004532728969470737
Confidence: 0.28813559322033894
Lift: 3.0228043143297376
=====================================
Rule: nan -> frozen vegetables
Support: 0.004799360085321957
Confidence: 0.20338983050847456
Lift: 3.094578333963626
=====================================
Rule: nan -> mineral water
Support: 0.007199040127982935
Confidence: 0.30508474576271183
Lift: 3.200616332819722
=====================================
Rule: nan -> frozen vegetables
Support: 0.005732568990801226
Confidence: 0.20574162679425836
Lift: 3.1303609383037156
=====================================
Rule: nan -> frozen vegetables
Support: 0.005999200106652446
Confidence: 0.21531100478468898
Lift: 3.0187810222242093
=====================================
Rule: nan -> frozen vegetables
Support: 0.006665777896280496
Confidence: 0.23923444976076558
Lift: 3.4980460188216425
=====================================
```

```
Rule: nan -> grated cheese
Support: 0.005332622317024397
Confidence: 0.3225806451612903
Lift: 3.283144395325426
=====================================
Rule: nan -> herb & pepper
Support: 0.006665777896280496
Confidence: 0.39062500000000006
Lift: 3.975682666214383
=====================================
Rule: nan -> herb & pepper
Support: 0.006399146780429276
Confidence: 0.3934426229508197
Lift: 4.004359721511667
=====================================
Rule: nan -> ground beef
Support: 0.004932675643247567
Confidence: 0.22424242424242427
Lift: 3.4118507591124225
=====================================
Rule: nan -> ground beef
Support: 0.005999200106652446
Confidence: 0.5232558139534884
Lift: 3.005315360233627
=====================================
Rule: nan -> spaghetti
Support: 0.007199040127982935
Confidence: 0.20300751879699247
Lift: 3.088761457396025
=====================================
Rule: nan -> mineral water
Support: 0.005199306759098787
Confidence: 0.22543352601156072
Lift: 3.429973384609973
=====================================
Rule: nan -> spaghetti
Support: 0.005065991201173177
Confidence: 0.20105820105820105
Lift: 3.0591025682303568
=====================================
Rule: mineral water -> frozen vegetables
Support: 0.004532728969470737
Confidence: 0.28813559322033894
Lift: 3.0228043143297376
=====================================
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota'
```

4.2 Sample code to do FP-Growth in Python

Definition: FP-Growth is a fast and efficient algorithm for mining frequent itemsets without generating candidate itemsets like Apriori. It compresses the dataset into a Frequent Pattern Tree (FP-Tree), which retains the itemset association information. Frequent itemsets are then extracted directly from the FP-Tree, making it faster for large datasets.

Algorithm:

1. Scan dataset to find frequent items (above minimum support).
2. Build FP-Tree to compress transactions while retaining item association.
3. Traverse the FP-Tree to extract frequent itemsets recursively.
4. Generate association rules from frequent itemsets.

Code:

```
 import pyfpgrowth
# Creating Sample Transactions
transactions = [
   ['Milk', 'Bread', 'Saffron'],
   ['Milk', 'Saffron'],
   ['Bread', 'Saffron', 'Wafer'],
   ['Bread', 'Wafer'],]
# Finding the frequent patterns with min support threshold=0.5
FrequentPatterns = pyfpgrowth.find_frequent_patterns(transactions=transactions,
support_threshold=0.5)
print(FrequentPatterns)
# Generating rules with min confidence threshold=0.5
print("Generating rules with min confidence threshold=0.5")
Rules = pyfpgrowth.generate_association_rules(patterns=FrequentPatterns,
confidence_threshold=0.5)
print(Rules)
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilafpgrowth.py
{('Milk',): 2, ('Milk', 'Saffron'): 2, ('Bread', 'Milk'): 1, ('Bread', 'Milk', 'Saffron'): 1, ('Wafer',): 2, ('Saffron', 'Wafer'): 1, ('Bread', 'Wafer'): 2
, ('Bread', 'Saffron', 'Wafer'): 1, ('Bread',): 3, ('Saffron',): 3, ('Bread', 'Saffron'): 2}
Generating rules with min confidence threshold=0.5
{('Milk',): (('Bread', 'Saffron'), 0.5), ('Saffron',): (('Bread',), 0.6666666666666666), ('Bread', 'Milk'): (('Saffron',), 1.0), ('Bread', 'Saffron'): ((('W
afer',), 0.5), ('Milk', 'Saffron'): (('Bread',), 0.5), ('Wafer',): (('Bread', 'Saffron'), 0.5), ('Bread',): (('Saffron',), 0.6666666666666666), ('Bread', '
Wafer'): (('Saffron',), 0.5), ('Saffron', 'Wafer'): (('Bread',), 1.0)}
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab>
```

LAB 5 : Classification (Using Naive Bayes Classifier)

Theory: Naive Bayes Classifier is a probabilistic machine learning algorithm based on Bayes' Theorem, used for classification tasks.It assumes that all features are independent of each other (naive assumption), which simplifies computation.For diabetes prediction, it calculates the probability of a patient having diabetes given their features (e.g., glucose level, BMI, age) and classifies them into Diabetic or Non-Diabetic.

Formula:

Bayes' Theorem : P(C|X)=P(X|C)*P(C) / P(X)

Where: C= Class label (Diabetic / Non-Diabetic)

- X = Feature vector (e.g., glucose, BMI, age)
- P(C|X) = Posterior probability of class C given features X
- P(X|C) = Likelihood of features X given class C
- P(C) = Prior probability of class C
- P(X) = Probability of features X

Code:
```
import pandas as pd
from sklearn import metrics
from sklearn.naive_bayes import GaussianNB
dataset = pd.read_csv('Diabetes.csv')
```

```
split = int(len(dataset)*0.7)
train, test = dataset[:split], dataset[split:]
p = train['Pragnency'].values
g = train['Glucose'].values
bp= train['Blod Pressure'].values
st= train['Skin Thikness'].values
ins= train['Insulin'].values
bmi= train['BMI'].values
dfp= train['DFP'].values
a= train['Age'].values
d= train['Diabetes'].values
trainfeatures=zip(p,g,bp,st,ins,bmi,dfp,a)
traininput=list(trainfeatures)
#print(traininput)
model = GaussianNB()
model.fit(traininput,d)
p = test['Pragnency'].values
g = test['Glucose'].values
bp= test['Blod Pressure'].values
st= test['Skin Thikness'].values
ins= test['Insulin'].values
bmi= test['BMI'].values
dpf= test['DFP'].values
a= test['Age'].values
d= test['Diabetes'].values
testfeatures=zip(p,g,bp,st,ins,bmi,dpf,a)
testinput=list(testfeatures)
predicted= model.predict(testinput)
print("Actual Class:   ", *d)
print("Predicted Class:", *predicted)
print("Confusion Matrix")
print(metrics.confusion_matrix(d, predicted))
print("********Classifiaction Measures*********")
print("Accuracy:",metrics.accuracy_score(d,predicted))
print("Recall:",metrics.recall_score(d,predicted))
print("Precision:",metrics.precision_score(d,predicted))
print("F1-Score:",metrics.f1_score(d,predicted))
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilanaivebayes.py
Actual Class:    0 0 1 1 1 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1 1 0 0 0 1 0 1 0 1 0 1 0 0 1 0 0 1 0 0 0 0 1 1 0
1 0 0 0 0 1 1 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 0 1 0 0 0 1 1 1 0 0 0 0 0 0 1 0 0 0 1 0 1 1 1 1 0 1 1 0 0 0 0 0 0 1 1 0 1 0 0 1 0 1
 0 0 0 0 0 1 0 1 0 1 0 1 1 0 0 0 0 1 1 0 0 0 1 0 1 1 0 0 1 0 0 1 1 0 0 1 0 0 1 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 0 1 1 0 0 1 0 0 1 0 1 1 1 0 0 1 1 1 0 1 0 1 0
1 0 0 0 0 1 0
Predicted Class: 0 0 1 1 0 1 0 0 1 1 0 1 1 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 1 0 1 0 1 0 1 0 1 0 0 1 1 1 0 0 1 0 0 0 0 1 1 0
1 0 1 0 0 1 1 0 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 1 1 0 0 0 0 0 0 1 0 1 1 0 1 1 1 1 0 0 0 0 0 1 1 0 1 1 0 1 1 0 0 0 0 1 0 0
 0 0 0 0 0 1 0 1 0 1 0 1 0 0 0 0 0 0 1 1 0 0 0 0 1 0 1 0 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 1 1 1 0 1 0 0 0 0 1 1 1 1 0 0 1 0
1 0 1 0 0 0 0
Confusion Matrix
[[128  24]
 [ 30  49]]
********Classifiaction Measures*********
Accuracy: 0.7662337662337663
Recall: 0.620253164556962
Precision: 0.6712328767123288
F1-Score: 0.6447368421052632
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab>
```

Lab 6: Clustering

6.1 : K-Means Clustering

Theory :K-Means is an unsupervised learning algorithm used to partition data into K distinct clusters based on feature similarity. It minimizes sum of squared distances between data points & their

corresponding cluster centroids.Widely used in customer segmentation, image compression, and pattern recognition.

Formula for Euclidean Distance (for assigning points to clusters):

$$d(x1, x2) = sqrt[(x2-x1)^2 - (y2-y1)^2$$

Algorithm:

-Initialize K cluster centroids randomly.

-Assign each data point to the nearest centroid using distance metric.

-Update centroids as the mean of points in each cluster.

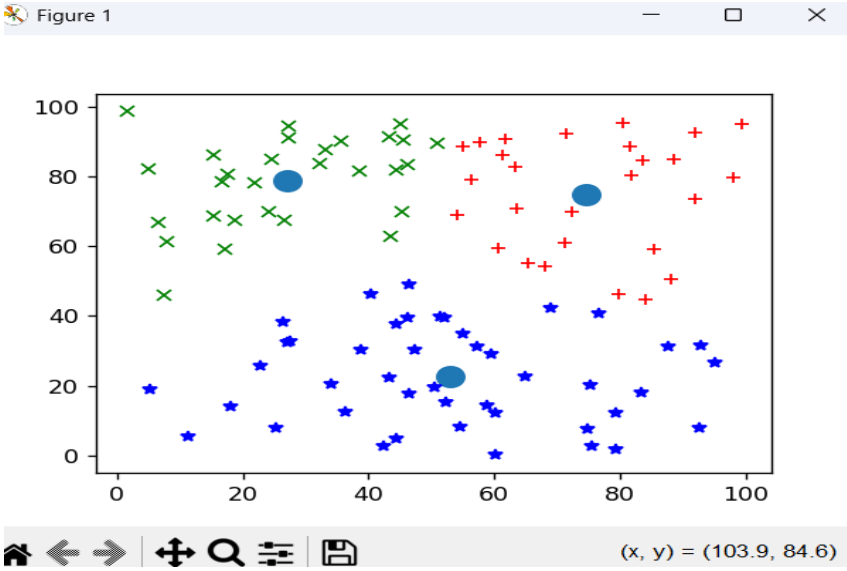
-Repeat steps 2–3 until centroids do not change (convergence).

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
data=100*np.random.rand(100,2)
print(*data)
km=KMeans(n_clusters=3,init='random')
km.fit(data)
centers = km.cluster_centers_
labels = km.labels_
print("Cluster Centers:",*centers)
print("Cluster Labels:",*labels)
colors = ["r","g","b"]
markers=["+","x","*"]
for i in range(len(data)):
    plt.plot(data[i][0], data[i][1], color=colors[labels[i]], marker=markers[labels[i]])
plt.scatter(centers[:, 0],centers[:, 1], marker = "o", s=50, linewidths = 5)
plt.show()
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilakmeans.py
[11.27270388  5.39259025] [88.08004477 50.69633148] [76.63231698 40.78640263] [60.65995291 59.34347136] [79.47909762 12.31505012] [72.46676146 69.74316278] [60.20246
4    0.13262317] [88.65269761 84.78232992] [95.23169852 26.586682  ] [26.45041933 38.29143585] [55.1453932  88.54093715] [63.35223784 82.86188092] [61.70782149 90.77
897485] [51.07127234 89.46243061] [27.56318364 32.71298779] [92.08696744 73.41943936] [59.5635997  29.08299741] [ 7.43043274 45.89184177] [60.3029911  12.07747696] [
35.76030114 90.28375433] [83.73988575 84.49696281] [54.14933598 69.04895691] [27.2096939  91.17758208] [43.62196271 62.70530851] [16.73535575 78.56585422] [45.221777
75 94.89418168] [34.15392759 20.42700301] [ 5.27845387 19.04293514] [27.3677772  94.42458891] [43.30557237 22.29595627] [17.6719724  80.47873459] [68.94106541 42.322
64258] [57.17731871 31.10376134] [ 7.99185169 61.3004263 ] [71.46410677 92.18918487] [81.84325109 80.37193895] [75.60473543  2.65642088] [47.31380446 30.25268032] [3
8.82911351 30.2701129 ] [92.66499621  7.83676248] [74.77442837  7.5916631 ] [68.17421477 54.14669706] [75.36492886 20.11075518] [79.35914129  1.68388371] [52.364973
  15.30212374] [85.3492189  59.02800058] [32.29324554 83.6348947 ] [15.44323094 86.11290324] [91.9734749  92.43711098] [43.23317155 91.20459835] [46.22292613 83.3411
534 ] [45.52106407 90.3765837 ] [50.48482837 19.61572687] [79.89429659 46.31664562] [18.06016876 14.00295014] [98.03229911 79.64144747] [33.07068635 87.66332921] [57
.81933393 89.70801326] [18.64902243 67.41472375] [ 5.07763659 82.08346088] [21.87324747 78.24593058] [38.51037432 81.56796893] [65.29063806 55.02468687] [25.28346337
  8.0484378 ] [26.6909396 67.5760728] [46.2750421 39.6318715] [24.2417931  70.01453315] [99.36425225 95.16487153] [44.3553908  37.61324405] [87.72577266 31.14750076]
 [24.55059757 84.77606391] [56.32364985 79.15564759] [ 1.55147999 98.67350377] [63.68089754 70.76560251] [81.5587998  88.72619662] [15.34089907 68.72706091] [36.3588
3311 12.68882091] [58.9517157  14.40571299] [17.19656686 59.30642678] [83.40618723 18.03153376] [55.04363995 34.81429287] [44.45462453 81.90681093] [52.00244101 39.4
1097031] [80.53715153 95.37792003] [46.47082133 49.11709655] [61.25076529 85.99681571] [ 6.53066492 66.91796919] [92.97169417 31.65450661] [64.96134996 22.51926221]
[71.15852621 60.99565129] [27.0420224  32.33684549] [51.40269455 39.70423557] [42.35843425  2.7254744 ] [54.6133481   8.10328063] [40.39827107 46.3371758 ] [46.50834
008 17.82536723] [84.13728952 44.80689156] [45.3779528  69.90723349] [44.48888865  4.7840922 ] [22.86360049 25.66443329]
Cluster Centers: [74.73678758 74.94688037] [27.10043177 78.91848016] [53.04222459 22.69222225]
Cluster Labels: 2 0 2 0 2 0 2 0 2 2 0 0 0 1 2 0 2 1 2 1 0 0 1 1 1 1 2 2 1 2 1 2 2 1 0 0 2 2 2 2 2 0 2 2 2 0 1 1 0 1 1 1 2 0 2 0 1 0 1 1 1 1 0 2 1 2 1 0 2 2 1 0 1 0 0
 1 2 2 1 2 2 1 2 0 2 0 1 2 2 0 2 2 2 2 2 2 0 1 2 2
```



6.2 Mini-Batch K-means Clustering

Theory : Mini-Batch K-Means is a faster variant of K-Means clustering that uses small random subsets (mini-batches) of the dataset to update cluster centroids.It reduces computation time and memory usage for large datasets while producing clusters similar to standard K-Means.Useful for big data applications like customer segmentation, image compression, and real-time clustering.

Algorithm:

-Initialize **K cluster centroids** randomly.

- Randomly select a **mini-batch** of data points.

-Assign mini-batch points to the **nearest centroid**.

-Update **centroids** based on mini-batch assignments.

-Repeat steps 2–4 until **convergence** or max iterations.

Code:

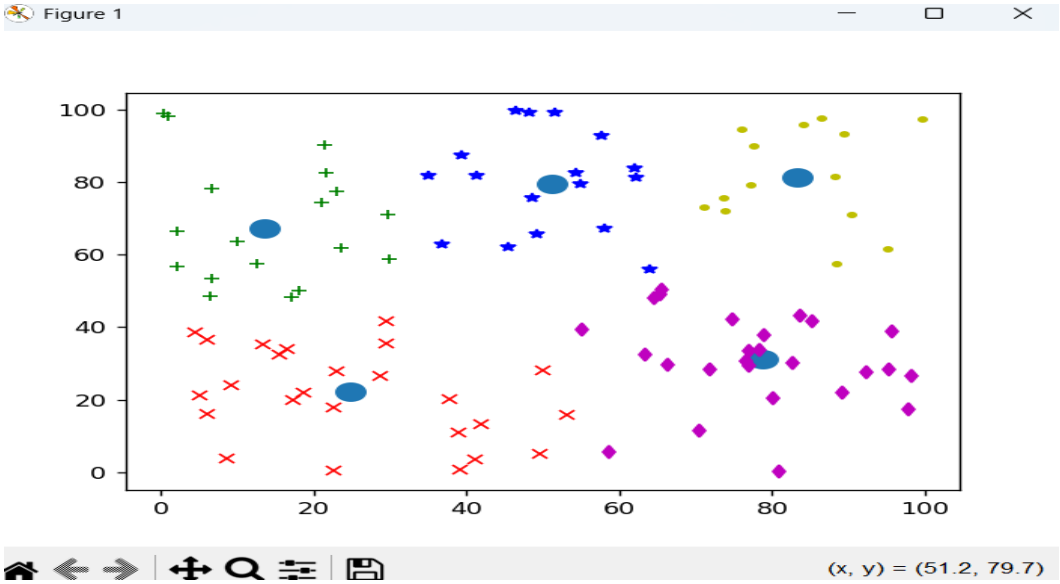
```
import time
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import MiniBatchKMeans
data=100*np.random.rand(100,2)
print(*data)
mbk=MiniBatchKMeans(n_clusters=5,init='random', batch_size=3000)
t0= time.time()
mbk.fit(data)
t1= time.time()
```

```
tt=t1-t0
print("Total Time:",tt)
cents = mbk.cluster_centers_
labels = mbk.labels_
print("Cluster Centers:",*cents)
print("Labels:",*labels)

colors = ["g","r","b",'y','m']
markers=["+","x","*",'.','d']
for i in range(len(data)):
    plt.plot(data[i][0], data[i][1], color=colors[labels[i]], marker=markers[labels[i]])
plt.scatter(cents[:, 0],cents[:, 1], marker = "o", s=50, linewidths = 5)
plt.show()
```

Output:

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilaminibatch.py
[82.69607889 30.07194334] [89.15455616 22.07725049] [29.91528137 58.80393917] [28.68959527 26.58927078] [41.92521997 13.34416515] [98.19040273 26.50457766] [74.87984
24  42.10683282] [ 6.17090279 36.54129226] [23.05851149 77.35300579] [65.43885951 48.93073949] [21.58565469 82.4467286 ] [95.12695002 61.59237654] [29.46121036 41.72
195335] [84.0043676  95.80474365] [18.14504968 50.00151186] [83.6773628  43.22482644] [ 0.8726343  98.05821885] [75.91375435 94.58717291] [90.3269761  70.96822592] [
57.6143615  92.83513681] [ 9.97230389 63.60617314] [64.5650637  48.08737108] [ 9.13715921 24.11324689] [58.0015397  67.18214901] [63.98061643 55.98044057] [92.400975
19 27.49263534] [ 5.15829655 21.2837031 ] [66.32373195 29.61826756] [73.59973036 75.72696584] [62.26448495 81.18505807] [80.14452746 20.41751237] [89.39441518 93.345
18646] [ 0.31941433 98.98449331] [45.45281911 61.96708051] [ 6.13422124 16.00371432] [23.68728441 61.83528066] [78.89813321 37.86203557] [13.3657684  35.13898383] [2
2.63881571 17.91129128] [41.18595109  3.54869812] [17.39293529 19.96241058] [38.90044288 10.97100185] [36.82615248 62.74239919] [63.32956286 32.43658043] [21.4378778
5 90.30245787] [ 4.53548034 38.5069875 ] [99.63957136 97.30313874] [48.54405534 75.59737294] [16.58144694 34.04960312] [85.25980107 41.57744722] [80.96999239  0.1310
8686] [95.22702313 28.43427301] [71.90011124 28.39204061] [58.70865728  5.69024664] [41.35080477 81.77038094] [ 2.12797003 66.50787309] [86.50479219 97.74094369] [37
.7410586 20.0967756] [22.59076443  0.40244779] [29.67726298 70.91570308] [65.55584445 50.20506721] [71.00732116 73.0878355 ] [23.09652973 27.91825075] [ 2.18398319 5
6.79247806] [39.12765703  0.68132011] [77.49823172 90.00028946] [88.24943432 81.50654782] [8.62126318 3.74984261] [17.20111556 48.35744604] [76.97883103 29.41951167]
 [55.16700376 39.22416437] [49.20258498 65.53644379] [70.38681814 11.51633365] [48.26327767 99.24657524] [95.73959875 38.80700357] [51.60984189 99.25099172] [39.4487
7421 87.35213564] [78.30863448 33.76552503] [15.4968317 32.3588763] [46.49498565 99.61348036] [49.95343028 28.0267927 ] [97.78242346 17.28913254] [ 6.52182382 48.525
82139] [53.19320244 15.77875492] [ 6.76916604 53.35523969] [73.78528968 71.9482741 ] [18.7656556  21.94923584] [54.9206459  79.46293857] [76.65624334 30.73548003] [7
7.1842326  79.15487213] [12.64629558 57.40625275] [54.31247581 82.62243302] [77.00886183 33.4406182 ] [62.04539905 83.81744303] [ 6.70538117 78.23462758] [88.3638081
  57.55053328] [35.0966038  81.81406077] [21.01474203 74.46261393] [29.49122706 35.52230497] [49.65803073  5.08921029]
Total Time: 2.0634801387786865
Cluster Centers: [13.65345719 67.10767306] [24.85801354 22.09344607] [51.26783263 79.34251192] [83.19943097 81.22174319] [78.74304145 31.10775188]
Labels: 4 4 0 1 1 4 4 1 0 4 0 3 1 3 0 4 0 3 3 2 0 4 1 2 2 4 1 4 3 2 4 3 0 2 1 0 4 1 1 1 1 1 2 4 0 1 3 2 1 4 4 4 4 4 2 0 3 1 1 0 4 3 1 0 1 3 3 1 0 4 4 2 4 2 4 2 2 4 1
 2 1 4 0 1 0 3 1 2 4 3 0 2 4 2 0 3 2 0 1 1
```



6.3 Hierarchical Clustering (Agglomerative Clustering)

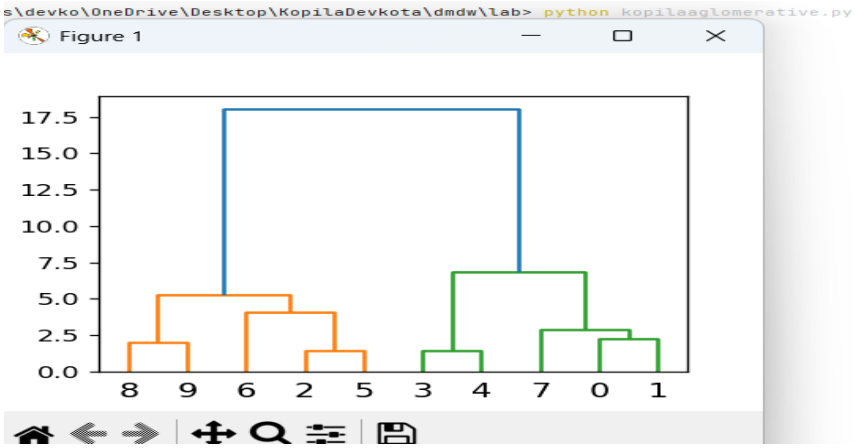
Theory : Hierarchical clustering is an unsupervised learning technique that builds a hierarchy of clusters either bottom-up (agglomerative) or top-down (divisive).Agglomerative clustering starts with each data point as its own cluster and merges the closest clusters iteratively until all points belong to a single cluster or a stopping criterion is met.

Algorithm:

1. Start with each data point as its own cluster.
2. Compute distance matrix between all clusters.
3. Merge the two closest clusters using a chosen linkage method.
4. Update the distance matrix after merging.
5. Repeat steps 3–4 until desired number of clusters or single cluster is formed.
6. Represent the result using a dendrogram or cluster labels.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
x = [4, 5, 10, 4, 3, 11, 14 , 6, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]
data = list(zip(x, y))
linkage_data = linkage(data, method='ward', metric='euclidean')
dendrogram(linkage_data)
plt.show()
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
x = [4, 5, 10, 4, 3, 11, 14 , 6, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]
data = list(zip(x, y))
hierarchical_cluster = AgglomerativeClustering(n_clusters=2, affinity='euclidean', linkage='ward')
labels = hierarchical_cluster.fit_predict(data)
print(labels)
plt.scatter(x, y, c=labels)
plt.show()
```



6.4 Clustering Iris data using KMedoids.

Theory : K-Medoids is an unsupervised clustering algorithm similar to K-Means but chooses actual data points (medoids) as cluster centers instead of mean points.It is more robust to outliers compared to K-Means because medoids are less influenced by extreme values.Commonly used in bioinformatics, image analysis, and customer segmentation.

Algorithm :

1. Select **K random medoids** from the dataset.
2. Assign each data point to the **nearest medoid**.
3. Compute the **total cost** (sum of distances from points to their medoids).
4. For each medoid, try **swapping with a non-medoid** point and compute new cost.
5. If cost decreases, **update medoids**.
6. Repeat steps 2–5 until **medoids do not change**.

Code:

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn import metrics
import matplotlib.pyplot as plt
from pyclustering.cluster.kmedoids import kmedoids
from pyclustering.utils import calculate_distance_matrix
import numpy as np
# Load iris dataset
iris_data = load_iris()
X = iris_data.data
y = iris_data.target
# Standardize
sc = StandardScaler()
X_scaled = sc.fit_transform(X)
# Create distance matrix
distance_matrix = calculate_distance_matrix(X_scaled)
# Choose initial medoids (random or fixed indexes)
initial_medoids = [0, 50, 100]
kmedoids_instance = kmedoids(distance_matrix, initial_medoids, data_type='distance_matrix')
kmedoids_instance.process()
# Get clusters and convert to labels
clusters = kmedoids_instance.get_clusters()
labels = np.zeros(len(X))
for cluster_id, cluster in enumerate(clusters):
    for idx in cluster:
        labels[idx] = cluster_id
print("Predicted Group:", labels)
# Evaluation
print("Homogeneity Score:", metrics.homogeneity_score(y, labels))
```

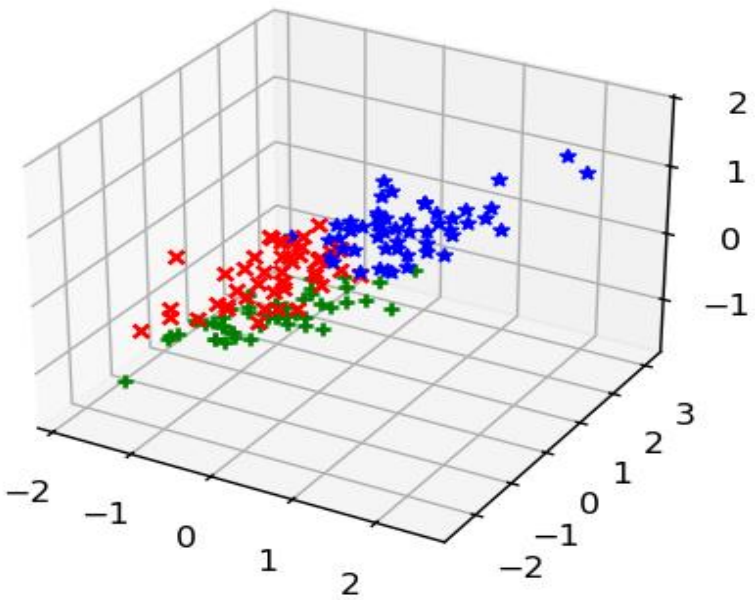
```
print("Silhouette Coefficient:", metrics.silhouette_score(X_scaled, labels))
# Plot
colors = ["g","r","b"]
markers=["+","x","*"]
fig = plt.figure(figsize=(12,8))
ax = fig.add_subplot(111, projection="3d")
for i in range(len(X_scaled)):
    ax.scatter(X_scaled[i][0], X_scaled[i][1], X_scaled[i][2],
    color=colors[int(labels[i])], marker=markers[int(labels[i])])
plt.show()
```

Output:

```
Python Console

PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilakmedoid.py
Predicted Group: [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 2. 2. 2. 1. 2. 1. 2. 1. 2. 1. 1. 1. 1. 1. 1. 2. 1. 1. 1. 1. 2. 1.
 1. 1. 1. 2. 2. 2. 1. 1. 1. 1. 1. 1. 1. 2. 2. 1. 1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 2. 1. 2. 2. 2. 2. 1. 2. 2. 2. 2. 2. 2. 1. 2. 2. 2. 2. 2. 1.
 2. 1. 2. 2. 2. 2. 2. 2. 2. 2. 2. 2. 2. 1. 1. 2. 2. 2. 2. 2. 2. 2. 1. 2.
 2. 2. 1. 2. 2. 1.]
Homogeneity Score: 0.6732054565880913
```

```
PS C:\Users\devko\OneDrive\Desktop\KopilaDevkota\dmdw\lab> python kopilakmedoid.py
Figure 1
```




```
x pane=−2.1881, y=−2.1638, z=0.9249
```